

Clase: OAuth 2.0 desde cero

Sep 30, 2026 · @Alex

OAuth 2.0 es el estándar que permite a una aplicación acceder a recursos de un usuario en otro servicio sin conocer su contraseña, usando permisos acotados y temporales llamados tokens.

1. El problema que resuelve

Autenticación vs. autorización

Son las dos palabras que más se confunden en entrevista:

- **Autenticación (AuthN):** comprobar *quién eres*. Ejemplo: iniciar sesión con usuario y contraseña.
- **Autorización (AuthZ):** decidir *qué puedes hacer*. Ejemplo: un analista puede ver solicitudes, pero solo un admin puede aprobarlas.

OAuth 2.0 es un protocolo de autorización, no de autenticación. Su trabajo es entregar permisos, no decir quién es el usuario. Para identidad existe OpenID Connect, que se construye encima de OAuth (sección 2).

Autorización delegada

El caso central de OAuth es la **autorización delegada**: tú (el dueño de los datos) le das a una aplicación de terceros permiso para actuar en tu nombre, sobre una parte de tus datos, por un tiempo limitado.

Tu app de consola quiere crear eventos en tu Google Calendar. Los datos son tuyos, viven en Google, y la app es de un tercero (tú como desarrollador, pero para Google es otra aplicación).

El antipatrón de la contraseña

Antes de OAuth, la solución era darle tu usuario y contraseña a la app. Se llamaba **password anti-pattern** y tenía cinco problemas:

1. La app guarda tu contraseña, normalmente en texto plano o reversible.
2. Obtiene acceso **total**: correo, Drive, pagos, todo, aunque solo necesitara el calendario.

3. No puedes revocar el acceso de una sola app sin cambiar la contraseña y romper todas las demás.
4. Si la app es hackeada, se filtra tu contraseña real.
5. No funciona con segundo factor (2FA) ni con inicio de sesión federado.

La analogía del valet parking

Algunos autos traen una **llave de valet**: enciende el motor y maneja pocos kilómetros, pero no abre la cajuela ni la guantera. OAuth es esa llave. En lugar de darle al tercero tu llave maestra (la contraseña), le das una llave limitada (el **token**), con permisos específicos (los **scopes**), que caduca y que puedes anular cuando quieras.

2. Cómo nació: de OAuth 1.0 a OAuth 2.1

OAuth nació en 2006 cuando desarrolladores de Twitter y Magnolia (entre ellos Blaine Cook y Chris Messina) buscaban cómo dejar que apps de terceros usaran sus APIs sin pedir contraseñas. Cada empresa tenía su propio mecanismo (Google AuthSub, Yahoo BBAuth, Flickr API Auth), así que decidieron crear un estándar abierto.

Año	Hito	Qué aportó
2007	OAuth Core 1.0	Primer estándar abierto. Cada petición se firmaba criptográficamente (HMAC-SHA1), incluso sobre HTTP sin cifrar
2010	RFC 5849	OAuth 1.0a publicado formalmente por la IETF. Corrigió un ataque de <i>session fixation</i>
2012	RFC 6749 y RFC 6750	OAuth 2.0 y los Bearer tokens . Eliminó las firmas y delegó la seguridad del transporte a HTTPS (TLS)
2014	OpenID Connect 1.0	Capa de identidad sobre OAuth 2.0: agrega el <code>id_token</code> (un JWT) para saber quién es el usuario
2015	RFC 7519 y RFC 7636	JWT como formato de token y PKCE , protección para apps móviles y de escritorio
2017	RFC 8252	Buenas prácticas para apps nativas: usar el navegador del sistema y redirección a <code>localhost</code>
2019	RFC 8628	<i>Device Authorization Grant</i> para TVs y dispositivos sin teclado

Año	Hito	Qué aportó
2023	RFC 9449	DPoP : tokens atados a una llave del cliente, para que un token robado no sirva
2025	RFC 9700	<i>Security Best Current Practice</i> : recopila una década de lecciones de seguridad
En curso	OAuth 2.1 (borrador)	Consolida 2.0 + las buenas prácticas: PKCE obligatorio, sin Implicit ni Password grant

Por qué OAuth 2.0 rompió con 1.0

OAuth 1.0 era seguro pero difícil. Firmar cada petición exigía ordenar parámetros, codificarlos exactamente igual que el servidor y calcular el HMAC; un espacio de más rompía todo. OAuth 2.0 lo simplificó: el token viaja tal cual en un header y HTTPS lo protege. A cambio, quedó como un **framework** flexible más que como un protocolo cerrado. Eran Hammer, editor principal, renunció en 2012 criticando justamente que era demasiado abierto y fácil de implementar mal. Esa crítica explica por qué después salieron tantos RFC de seguridad y por qué existe OAuth 2.1.

Qué es OpenID Connect (OIDC)

OAuth 2.0 da permisos, pero muchas empresas lo usaban para "iniciar sesión con Facebook", algo para lo que no fue diseñado. OIDC formalizó ese uso: agrega el scope `openid`, el `id_token` con datos del usuario (`sub`, `email`, `name`) y el endpoint `/userinfo`. Regla para entrevista: **OAuth = autorización, OIDC = autenticación sobre OAuth**.

3. Los 4 roles y la definición formal

Todo en OAuth gira en torno a cuatro roles. Aprenderlos en inglés es importante porque así aparecen en documentación y entrevistas.

Rol	Qué es	En tu proyecto de Google Calendar
Resource Owner	Quien es dueño de los datos y puede dar permiso. Normalmente el usuario final	Tú, con tu cuenta <code>a.ortega8897@gmail.com</code>

Rol	Qué es	En tu proyecto de Google Calendar
Client	La aplicación que quiere acceder a los datos en nombre del dueño	Tu consola en C#
Authorization Server (AS)	El servidor que autentica al dueño, pide su consentimiento y emite tokens	<code>accounts.google.com</code> y <code>oauth2.googleapis.com</code>
Resource Server (RS)	La API que guarda los datos y acepta tokens para servirlos	<code>www.googleapis.com/calendar/v3</code>

El Authorization Server y el Resource Server pueden ser el mismo sistema o estar separados. En empresas grandes suelen estar separados: un servidor de identidad central (Microsoft Entra ID, Auth0, Okta, Keycloak) y muchas APIs que confían en él.

La definición establecida

El RFC 6749 lo define así (traducción propia):

"El framework de autorización OAuth 2.0 permite que una aplicación de terceros obtenga acceso limitado a un servicio HTTP, ya sea en nombre de un dueño del recurso, orquestando una interacción de aprobación entre el dueño y el servicio HTTP, o permitiendo que la aplicación obtenga acceso en su propio nombre."

Desarmada frase por frase:

- **Framework de autorización:** no es un solo flujo, es un conjunto de piezas y reglas que se combinan según el caso.
- **Aplicación de terceros:** el Client.
- **Acceso limitado:** acotado por scopes y por tiempo de expiración.
- **Servicio HTTP:** el Resource Server; OAuth vive sobre HTTP.
- **En nombre de un dueño del recurso:** autorización delegada, con consentimiento (Authorization Code).
- **O en su propio nombre:** máquina a máquina, sin usuario (Client Credentials).

Tu versión para entrevista, en una frase: "OAuth 2.0 es un framework de autorización que permite a una aplicación obtener acceso limitado y temporal a una API, en nombre de un usuario o en nombre propio, mediante tokens emitidos por un servidor de autorización, sin compartir credenciales."

4. Terminología completa

Tokens

Término	Qué es	Detalle clave
Access token	La credencial que el Client presenta a la API en cada petición	Vida corta (en Google, 1 hora). Viaja en el header <code>Authorization: Bearer <token></code>
Refresh token	Credencial para pedir nuevos access tokens sin volver a molestar al usuario	Vida larga. Solo se envía al Authorization Server, nunca a la API. Debe guardarse cifrado
ID token	Token de OpenID Connect con la identidad del usuario	Siempre es un JWT. Es para el Client, no para llamar APIs
Authorization code	Código temporal que el AS entrega tras el consentimiento	Dura segundos o minutos y es de un solo uso. Se cambia por tokens
Bearer token	Tipo de token donde "quien lo porta, lo usa"	Como un billete: si te lo roban, lo pueden gastar. Por eso exige HTTPS
Opaque token	Token que es una cadena sin significado para el Client	La API lo valida preguntándole al AS (<i>introspection</i> , RFC 7662)
JWT (JSON Web Token)	Token autocontenido: <code>header.payload.signature</code> en Base64URL	La API lo valida localmente verificando la firma con la llave pública del AS (JWKS)

Por qué dos tokens y no uno: el access token viaja muchísimo (cada petición), así que se hace de vida corta para que un robo dure poco. El refresh token casi no viaja y solo va al AS, que puede revocarlo. Es un balance entre seguridad y comodidad.

Claims de un JWT

Un *claim* es un dato dentro del payload. Los estándar: `iss` (quién lo emitió), `sub` (el sujeto, el id del usuario), `aud` (para qué API es), `exp` (cuándo expira), `iat` (cuándo se emitió), `scope` (permisos). **Un JWT está firmado, no cifrado:** cualquiera puede leer su contenido en jwt.io, así que nunca lleva datos sensibles.

Scopes y consentimiento

- **Scope:** un permiso con nombre. En Google, `https://www.googleapis.com/auth/calendar.events` permite manejar eventos, y `.../auth/calendar.readonly` solo leer.
- **Principio de mínimo privilegio:** pide solo los scopes que necesitas. Si tu token se filtra, el daño queda acotado, y la pantalla de consentimiento asusta menos al usuario.
- **Consent screen:** la pantalla donde el usuario ve qué app pide qué permisos y acepta o rechaza.

Tipos de Client

- **Confidential client:** puede guardar un secreto de forma segura porque corre en un servidor. Ejemplo: tu backend de ASP.NET.
- **Public client:** no puede guardar secretos porque su código está en manos del usuario. Ejemplos: una SPA en React, una app móvil y **tu consola de escritorio**. Por eso el `client_secret` de una Desktop app no se considera realmente secreto y la protección real viene de PKCE.
- **Client ID:** identificador público de la app. **Client secret:** contraseña de la app, solo para confidential clients.

Endpoints y parámetros

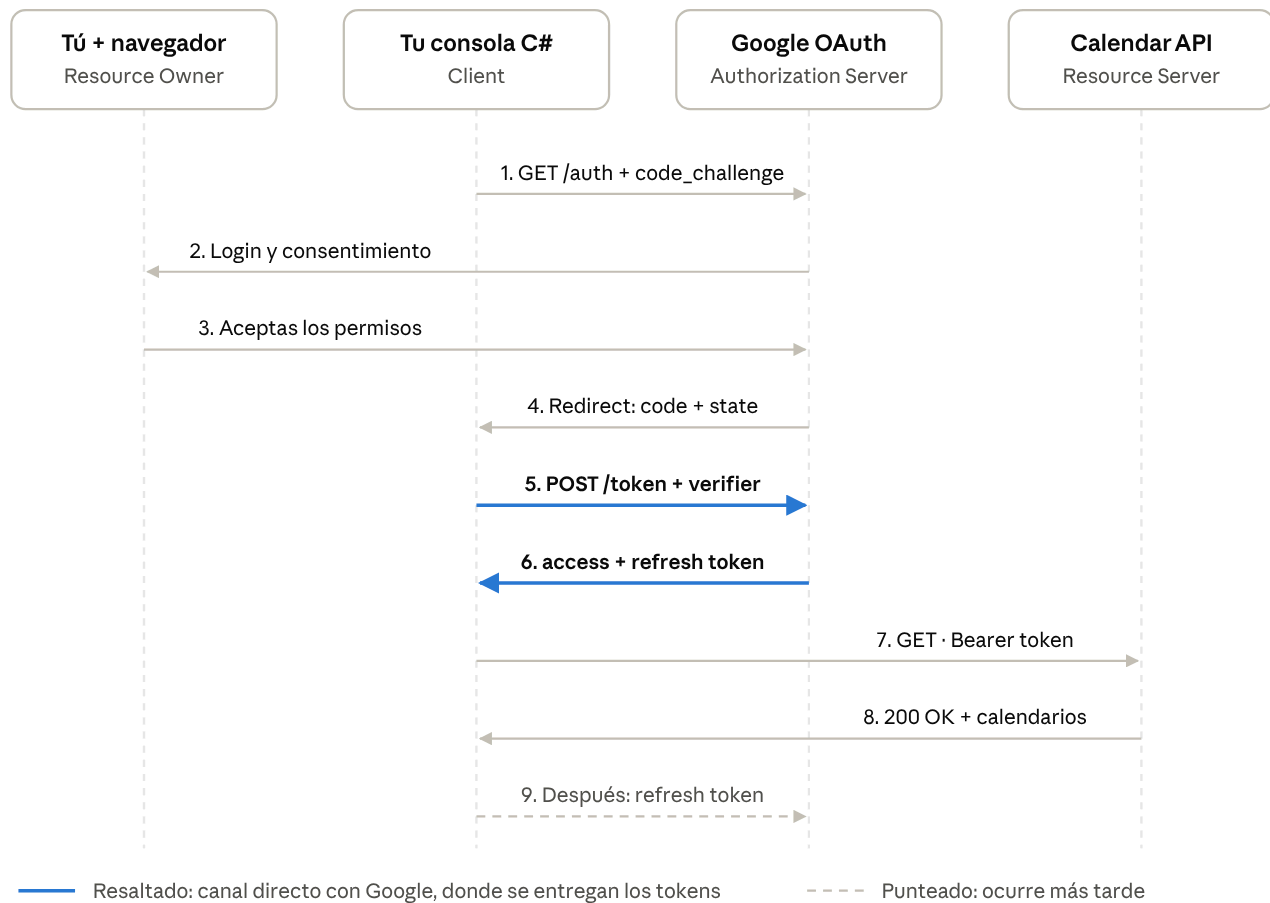
Término	Qué es
Authorization endpoint	URL del AS que se abre en el navegador para login y consentimiento. En Google: <code>accounts.google.com/o/oauth2/v2/auth</code>

Término	Qué es
Token endpoint	URL del AS donde el Client cambia el code o el refresh token por tokens. Es una petición POST de servidor a servidor. En Google: <code>oauth2.googleapis.com/token</code>
Redirect URI (callback)	URL a la que el AS regresa al usuario con el code. Debe estar registrada exactamente igual; si no, el AS rechaza la petición
<code>response_type=code</code>	Pide un authorization code
<code>grant_type</code>	En el token endpoint, qué estás canjeando: <code>authorization_code</code> , <code>refresh_token</code> , <code>client_credentials</code> ...
<code>state</code>	Valor aleatorio que el Client envía y verifica al regreso. Protege contra CSRF
PKCE (<code>code_verifier</code> / <code>code_challenge</code>)	El Client inventa un secreto aleatorio, envía su hash al inicio y el secreto original al canjear. Así un code interceptado no sirve
Discovery document	JSON público en <code>/.well-known/openid-configuration</code> que lista todos los endpoints del AS
Revocation endpoint	URL para invalidar un token (RFC 7009)

5. El flujo principal: Authorization Code con PKCE

Este es el flujo que vas a programar. Hay dos canales: el **front channel** (el navegador, donde viaja el code) y el **back channel** (petición directa de tu consola a Google, donde viajan los tokens).

El code pasa por el navegador; los tokens solo viajan de tu consola a Google



Authorization Code flow con PKCE · 4 roles, 9 pasos

- 1. Preparación.** Tu consola genera un `code_verifier` aleatorio, calcula su hash SHA-256 (`code_challenge`) y genera un `state` aleatorio. Abre un servidor local en `http://127.0.0.1:<puerto>` para recibir la respuesta.
- 2. Petición de autorización.** Abre el navegador en el authorization endpoint con `client_id`, `redirect_uri`, `response_type=code`, `scope`, `state` y `code_challenge`.
- 3. Login y consentimiento.** Google te autentica (contraseña, 2FA) y te muestra qué permisos pide la app.
- 4. Redirección con el code.** Google redirige el navegador a tu `redirect_uri` con `?code=...&state=...`. Tu consola verifica que el `state` sea el mismo que envió.
- 5. Canje del code.** Tu consola hace un POST al token endpoint con `grant_type=authorization_code`, el `code`, el `code_verifier`, el `client_id` y el `redirect_uri`.

6. **Emisión de tokens.** Google calcula el hash del `code_verifier`, lo compara con el `code_challenge` del paso 2 y, si coincide, responde con `access_token`, `refresh_token`, `expires_in` y `scope`.

7. **Llamada a la API.** Tu consola llama a Calendar con `Authorization: Bearer <access_token>`.

8. **Respuesta.** La API valida el token y el scope, y devuelve los datos.

9. **Renovación.** Cuando el access token expira, tu consola hace POST al token endpoint con `grant_type=refresh_token` y obtiene uno nuevo, sin abrir el navegador.

La librería `Google.Apis.Auth` hace los pasos 1 a 6 y el 9 por ti con `GoogleWebAuthorizationBroker`. Aun así, en entrevista debes poder narrarlos.

6. Los grant types: cuál usar y cuándo

Un **grant type** (tipo de concesión) es la forma en que el Client obtiene el token. Elegir el correcto es una pregunta típica de entrevista.

Grant type	Hay usuario	Cuándo se usa	Estado
Authorization Code + PKCE	Sí	Apps web, SPAs, móviles y de escritorio que actúan en nombre de un usuario. Tu caso	Recomendado para todo lo que tenga usuario
Client Credentials	No	Máquina a máquina: un servicio backend llama a otra API en nombre propio. Ejemplo: un job nocturno que consulta un buró de crédito	Recomendado para servicios
Refresh Token	Ya dio permiso	Renovar el access token cuando expira	Estándar

Grant type	Hay usuario	Cuándo se usa	Estado
Device Authorization	Sí, en otro dispositivo	TVs, consolas y CLIs sin navegador: muestran un código y el usuario lo aprueba en su celular	Recomendado para ese caso
Implicit	Sí	Antes se usaba en SPAs: el token llegaba directo en la URL	Obsoleto , eliminado en OAuth 2.1: el token quedaba expuesto en el historial y logs
Resource Owner Password Credentials (ROPC)	Sí	La app pide usuario y contraseña y los cambia por un token	Obsoleto : repite el antipatrón de la contraseña

Regla rápida: ¿hay usuario? Authorization Code + PKCE. ¿No hay usuario? Client Credentials. ¿No hay navegador? Device flow.

También existen extensiones como **Token Exchange** (RFC 8693), para que un servicio cambie un token por otro al llamar a un servicio interno, y **JWT Bearer / assertion grants**, que usan las *service accounts* de Google: el servicio firma un JWT con su llave privada y lo cambia por un access token.

7. Alternativas y cuándo conviene cada una

OAuth no siempre es la respuesta. Si solo tu backend llama a una API con una cuenta propia, suele bastar algo más simple.

Mecanismo	Cómo funciona	Ventaja	Desventaja	Uso típico
API key	Una cadena fija en un header (<code>x-api-key</code>) o en la URL	Muy simple	No identifica usuarios, no expira sola, suele dar acceso total	Servicios de servidor a servidor, APIs públicas con cuota (Google Maps)
HTTP Basic	<code>Authorization: Basic base64(usuario:contraseña)</code>	Estándar y simple	Base64 no es cifrado; la credencial viaja en cada petición	APIs internas, Twilio (Account SID + Auth Token)

Mecanismo	Cómo funciona	Ventaja	Desventaja	Uso típico
Sesión con cookie	El servidor guarda la sesión y el navegador envía una cookie	Nativo en apps web; fácil de invalidar	Atado al navegador y a un dominio; requiere protección CSRF	Apps MVC clásicas, como Forms Authentication o ASP.NET Identity
SAML 2.0	El proveedor de identidad envía una aserción XML firmada al navegador	Maduro en empresas	Pesado (XML), pensado para web y no para APIs ni móviles	Single Sign-On corporativo
Firma HMAC	Cada petición se firma con un secreto compartido	El secreto nunca viaja; detecta alteraciones	Implementación delicada	AWS Signature V4, validación de webhooks (Twilio, Meta)
mTLS	Cliente y servidor se autentican con certificados	Muy seguro	Gestión de certificados compleja	Banca, Open Banking, redes internas
Kerberos / NTLM	Tickets dentro de un dominio Windows	Transparente en la red corporativa	Solo funciona dentro de la red	<i>Windows Authentication</i> en IIS e intranets

Cómo elegir: OAuth se vuelve necesario cuando hay **terceros** accediendo a datos de **usuarios**, cuando necesitas **permisos granulares y revocables**, o cuando varias apps comparten un servidor de identidad (SSO con OIDC). Para una integración propia de servidor a servidor, una API key bien guardada o Client Credentials suelen ser suficientes.

8. Dónde se usa

Contextos comunes

- **"Iniciar sesión con Google / Microsoft / Apple":** OIDC sobre OAuth.
- **Integraciones de terceros:** Zapier leyendo tu Gmail, Slack conectado a Google Drive, apps de contabilidad conectadas al banco.
- **SSO corporativo:** Microsoft Entra ID, Okta o Auth0 emiten tokens para todas las apps internas.
- **Microservicios:** un gateway valida JWTs y los servicios se llaman entre sí con Client Credentials.

- **Open Banking / Open Finance:** los bancos exponen APIs protegidas con perfiles endurecidos de OAuth (FAPI, con mTLS o tokens atados al cliente). En México, la Ley Fintech (art. 76) contempla APIs abiertas entre instituciones, así que es un tema relevante para fintech.
- **En .NET:** ASP.NET Core trae `AddAuthentication().AddJwtBearer()` para proteger APIs y `AddOpenIdConnect()` para login. Duende IdentityServer y OpenIddict sirven para montar tu propio Authorization Server.

Tu experiencia en DiSí, con nombre técnico

Ya trabajaste con varios de estos mecanismos. Esta tabla te sirve para contarle en entrevista; verifica en el código de DiSí cómo se autentica cada una.

Integración	Mecanismo probable	Cómo lo dirías
Twilio (cobranza)	HTTP Basic con Account SID + Auth Token; webhooks firmados con HMAC (<code>X-Twilio-Signature</code>)	"Autenticación Basic en las llamadas salientes y validación de firma HMAC en los webhooks entrantes"
WhatsApp Cloud API (onboarding)	Bearer token de Meta (token de usuario del sistema); webhooks con <code>X-Hub-Signature-256</code>	"Bearer token emitido por Meta y verificación de firma SHA-256 en cada webhook"
Google Maps (originación)	API key restringida por dominio o IP	"API key con restricciones de dominio y de API"
Ekatena y Kobra	Revísalo: API key, Basic o Client Credentials	Prepara una frase como las anteriores
Portal del cliente	Probablemente sesión con cookie en MVC	"Autenticación por cookie de sesión y autorización por roles"

La diferencia es que en esas integraciones tu servidor usaba **su propia cuenta**. Con Google Calendar vas a actuar **en nombre de un usuario**, y eso es lo que hace necesario OAuth.

9. Seguridad y errores comunes

Riesgo	Qué pasa	Cómo se previene
Robo del authorization code	Otra app intercepta el code en la redirección	PKCE en todos los clientes
CSRF en el callback	Un atacante hace que tu app acepte un code suyo	Parámetro state aleatorio y verificado
Open redirect	El AS redirige el code a una URL del atacante	Redirect URIs registradas con coincidencia exacta
Token filtrado	Alguien lo usa como si fuera el dueño (es un bearer token)	Vida corta, HTTPS siempre, nunca en la URL ni en logs; DPoP o mTLS en casos críticos
Refresh token robado	Acceso de larga duración	Guardarlo cifrado (DPAPI en Windows, Azure Key Vault); rotación : cada uso emite uno nuevo e invalida el anterior
Secretos en el repositorio	<code>credentials.json</code> o tokens en GitHub	<code>.gitignore</code> , User Secrets de .NET, variables de entorno o un vault
Validar mal un JWT	Aceptar tokens falsos, expirados o de otra API	Verificar firma, <code>iss</code> , <code>aud</code> y <code>exp</code> ; rechazar <code>alg: none</code>
Scopes excesivos	Daño amplio si algo se filtra	Mínimo privilegio
Usar el access token como prueba de identidad	Otra app podría reutilizar un token emitido para ella	Para identidad, usa el <code>id_token</code> de OIDC y valida su <code>aud</code>

Códigos HTTP que vas a ver

- **400 `invalid_grant`**: el code o el refresh token ya se usó, expiró o fue revocado. En Google pasa si la app está en modo *Testing*: los refresh tokens caducan a los 7 días.
- **401 Unauthorized**: falta el token, es inválido o expiró. Tu Client debe renovar el token y reintentar una vez.

- **403 Forbidden:** el token es válido, pero no tiene el scope o el permiso necesario.
- **redirect_uri_mismatch** : la redirect URI no coincide con la registrada.

La diferencia entre 401 y 403 se pregunta mucho: **401 = no sé quién eres o tu credencial no sirve; 403 = sé quién eres, pero no tienes permiso.**

10. Preguntas de entrevista para autoevaluarte

Contesta en voz alta, sin mirar el documento. Márcala cuando puedas explicarla en menos de un minuto.

Básicas

- ☐ ¿Cuál es la diferencia entre autenticación y autorización? ¿Cuál resuelve OAuth?
- ☐ Nombra los 4 roles de OAuth y di cuál es cada uno en una integración con Google Calendar.
- ☐ ¿Por qué existen access token y refresh token?
- ☐ ¿Qué es un scope y por qué aplicar mínimo privilegio?
- ☐ ¿Qué diferencia hay entre OAuth 2.0 y OpenID Connect?

Intermedias

- ☐ Explica el Authorization Code flow paso a paso.
- ☐ ¿Qué problema resuelve PKCE y cómo funciona?
- ☐ ¿Para qué sirve el parámetro `state` ?
- ☐ ¿Qué grant type usarías para un job nocturno sin usuario? ¿Y para una SPA en React?
- ☐ ¿Qué diferencia hay entre un confidential client y un public client?
- ☐ ¿Por qué el Implicit grant se considera obsoleto?

Mid-senior

- ☐ ¿Cómo valida una API en ASP.NET Core un JWT? ¿Qué claims revisas?
- ☐ ¿Token opaco o JWT? Ventajas y desventajas de cada uno para revocar acceso.
- ☐ ¿Cómo guardarías refresh tokens en un backend multiusuario?
- ☐ ¿Cuándo usarías una API key en lugar de OAuth?
- ☐ Un cliente reporta errores 401 intermitentes. ¿Cómo lo diagnosticas?
- ☐ ¿Qué diferencia hay entre 401 y 403?

Fuentes

- [RFC 6749: The OAuth 2.0 Authorization Framework](#)
- [RFC 6750: Bearer Token Usage](#)
- [RFC 7636: PKCE](#)
- [RFC 8252: OAuth 2.0 for Native Apps](#)
- [RFC 9700: OAuth 2.0 Security Best Current Practice](#)
- [OAuth 2.1 \(borrador de la IETF\)](#)
- [OpenID Connect Core 1.0](#)
- [Google: OAuth 2.0 para apps de escritorio](#)
- [oauth.net](#), referencia de la comunidad